COMPUTER ENGINEERING
QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structures and Algorithms	**SECTION:**	CPE21S2
DATE PERFORMED:	07/17/2026	**DATE SUBMITTED:**	07/21/2026
NAME:	Realubit, Seth Andrew I.	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. 2 **ARRAYS, POINTERS AND DYNAMIC MEMORY ALLOCATION**

1. PROCEDURE OUTPUT

What procedures did you perform?

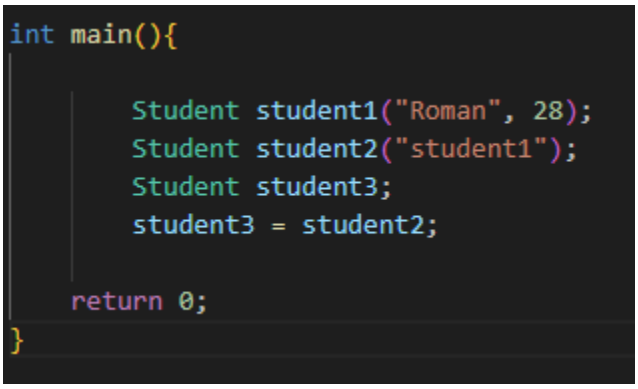
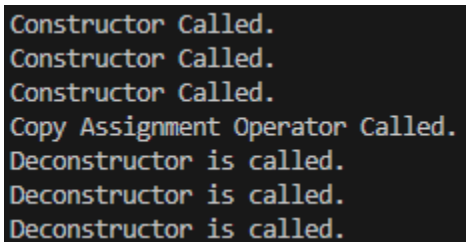
Screenshot	 *Figure 1. Code Snippet of int main()*  *Figure 2. Code Snippet of terminal*
Observation	I noticed that in figure 2, the constructor and deconstructor were called multiple times while running the code. When put in the code visualizer I noticed that the 3rd line from int main stores the string “Roman” into student 3 before eventually going to student 1. The same followed until John Doe was put in student 3 along with the age. This is when the copy assignment operator was called in copying the strings then was one-by-one deleted by the deconstructor but the ages remained.

Table 2-1. Initial Driver Program

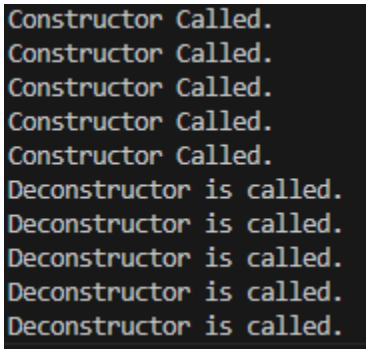
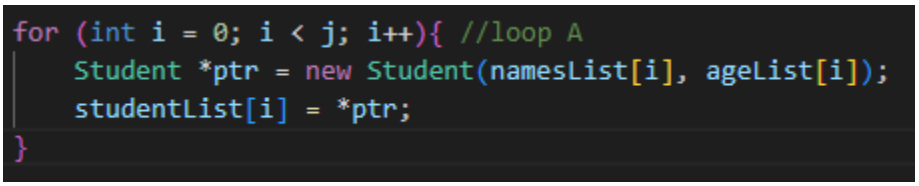
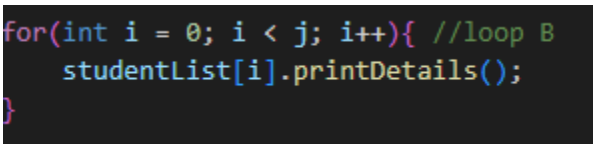
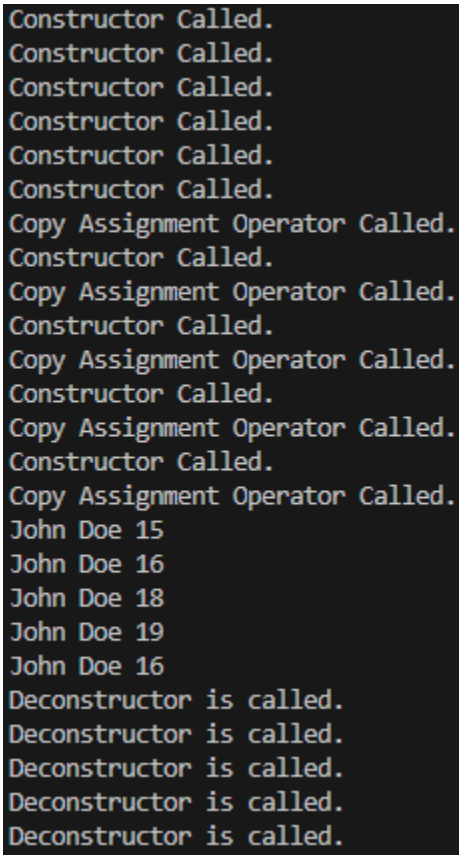
Screenshot	 *Figure 3. Code Snippet of int main()*  *Figure 4. Code Snippet of output in terminal.*
Observation	From my observation in the code visualizer, the studentList slowly gets filled with the information and age of John Doe. Moving on from that is when the namesList gets placed inside it, followed by the ageList array. The output constructor was for placing the name John Doe with his/her age inside the array named studentList. Meanwhile, the deconstructor slowly removed what information inside the arrays until only the age of John Doe was left.

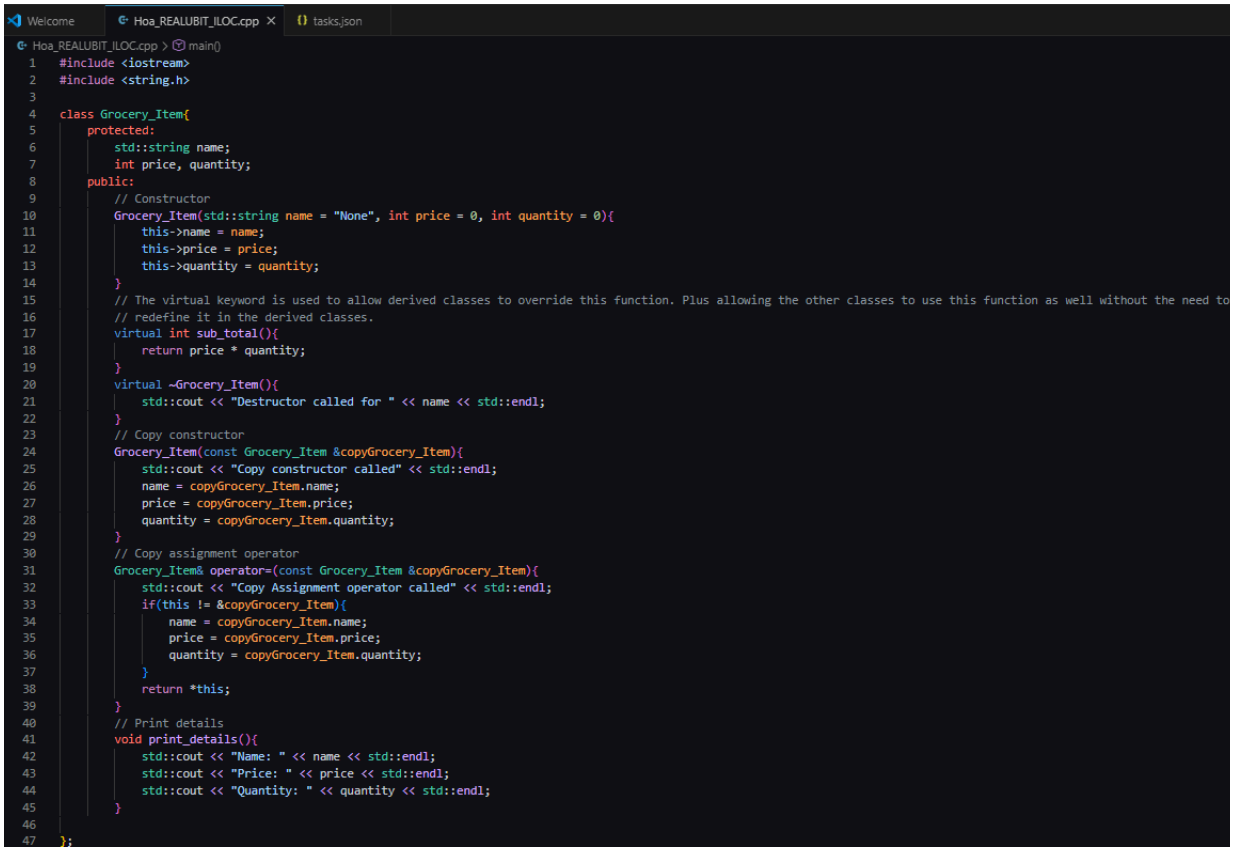
Table 2-2. Modified Student Driver with Student Lists

Loop A	 *Figure 5. Code Snippet of for loop A*
Observation	For figure 5, loop A repeatedly runs the copy assignment constructor and constructor and eventually change the studentList into the different ages and

Loop B	 *Figure 6. Code Snippet of for loop B*
Observation	From my understanding of figure 6, loop B, it repeatedly executes all the information inside the studentList which repeatedly changes with the name and ages of John Doe.
Output	 *Figure 7. Code Snippet of the output on the terminal*
Observation	From what I noticed with the output on the terminal, the constructor is called until the student list is filled with John Doe and his age. Following that the copy assignment operator and constructor was for filling the ages of the student list from the ageList. The multiple john doe and his/her age was from the second loop and the deconstructor was for deleting some of the information inside the code.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.



```cpp
#include <iostream>
#include <string.h>

class Grocery_Item{
    protected:
        std::string name;
        int price, quantity;
    public:
        // Constructor
        Grocery_Item(std::string name = "None", int price = 0, int quantity = 0){
            this->name = name;
            this->price = price;
            this->quantity = quantity;
        }
        // The virtual keyword is used to allow derived classes to override this function. Plus allowing the other classes to use this function as well without the need to
        // redefine it in the derived classes.
        virtual int sub_total(){
            return price * quantity;
        }
        virtual ~Grocery_Item(){
            std::cout << "Destructor called for " << name << std::endl;
        }
        // Copy constructor
        Grocery_Item(const Grocery_Item &copyGrocery_Item){
            std::cout << "Copy constructor called" << std::endl;
            name = copyGrocery_Item.name;
            price = copyGrocery_Item.price;
            quantity = copyGrocery_Item.quantity;
        }
        // Copy assignment operator
        Grocery_Item& operator=(const Grocery_Item &copyGrocery_Item){
            std::cout << "Copy Assignment operator called" << std::endl;
            if(this != &copyGrocery_Item){
                name = copyGrocery_Item.name;
                price = copyGrocery_Item.price;
                quantity = copyGrocery_Item.quantity;
            }
            return *this;
        }
        // Print details
        void print_details(){
            std::cout << "Name: " << name << std::endl;
            std::cout << "Price: " << price << std::endl;
            std::cout << "Quantity: " << quantity << std::endl;
        }

};
```

Figure 8. Code Snippet of Grocery_Item base class

Firstly, I chose to use a base class like Grocery_item since at first, I only made the base class Fruit and Vegetable for the whole code, but I ran into a problem on how I should add both the fruit and vegetable inside the array without making a separate one for each. Instead of simply that, I tried using inheritance on C++, while not covered yet; I had AI explain the concept of OOP in the C++ language because of the thought that it would be entirely a different syntax. This led to discovering “virtual” along with my solution to the supplementary activity. The reason for the use of virtual is simply because, in Python, each method is dynamic, meaning it checks the actual class before running, while in C++, it needs to be marked as virtual. Meanwhile, in the other parts of the code, some were just copied from the activity, like the cop constructor and copy assignment constructor, with some slight modifications.

```cpp
class Fruit : public Grocery_Item{
    public:
        // Constructor
        Fruit(std::string name = "None", int price = 0, int quantity = 0) : Grocery_Item(name, price, quantity) {}
        // Copy constructor
        Fruit(const Fruit &copyFruit) : Grocery_Item(copyFruit){
            std::cout << "Copy constructor called" << std::endl;
        }
        // Copy assignment operator
        Fruit& operator=(const Fruit &copyFruit){
            std::cout << "Copy Assignment operator called" << std::endl;
            Grocery_Item::operator=(copyFruit);
            return *this;
        }
        // The deconstructor was removed due to already having the grocery item destructor in the base class.
};

class Vegetable : public Grocery_Item{
    public:
        // Constructor
        Vegetable(std::string name = "None", int price = 0, int quantity = 0) : Grocery_Item(name, price, quantity) {}
        // Copy constructor
        Vegetable(const Vegetable &copyVegetable) : Grocery_Item(copyVegetable){
            std::cout << "Copy constructor called" << std::endl;
        }
        // Copy assignment operator
        Vegetable& operator=(const Vegetable &copyVegetable){
            std::cout << "Copy Assignment operator called" << std::endl;
            Grocery_Item::operator=(copyVegetable);
            return *this;
        }
};
```

Figure 9. Code Snippet derived class, Fruit and Vegetable

In Figure 9, it can be directly seen where the inheritance comes from in Python; in Python, the syntax for the derived class to be inherited can just be “class Fruit(Grocery_Item)”, while it requires the syntax in the figure to actually inherit the public from Grocery_Item. Next, as seen in the figure, the constructor and the big three are missing due to this being already called inside the base class. It can also be seen that there is no sub-total or the deconstructor itself since it would just be redone. While I haven’t tested the code if there is actually one inside the derived class. I assume it would run into an error. Lastly, there is no private in the derived classes since it is already inside the base class, which is protected and only used when needed.

```
81  int Total_sum(Grocery_Item *GroceryList[], int size){
82      int total = 0;
83      for (int i = 0; i < size; i++){
84          if (GroceryList[i] != nullptr){
85              total += GroceryList[i]->sub_total();
86          }
87      }
88      return total;
89  }
```

Figure 10. Code Snippet of Total_sum function

The total_sum function is just a simple for loop, on line 81 it is noticeable that the size of the GroceryList[] is blank, while 4 can be written inside; it is negligible based on my tests and proved to be the same either way. Next, if seen on line 84, there is a nullptr, this can be seen on the next figure as well. This is simply a safety check and to skip any deleted slots inside the array, like lettuce in problem 4.

```
91  int main(){
92      Grocery_Item *GroceryList[4];
93      GroceryList[0] = new Fruit("Apple", 10, 7);
94      GroceryList[1] = new Fruit("Banana", 10, 8);
95      GroceryList[2] = new Vegetable("Broccoli", 60, 12);
96      GroceryList[3] = new Vegetable("Lettuce", 50, 10);
97
98      for (int i = 0; i < 4; i++){
99          GroceryList[i]->print_details();
100         std::cout << "Subtotal: " << GroceryList[i]->sub_total() << std::endl;
101     }
102
103     int total = Total_sum(GroceryList, 4);
104     std::cout << "Total: " << total << std::endl;
105
106     delete GroceryList[3];
107     GroceryList[3] = nullptr;
108     std::cout << "Lettuce has been deleted." << std::endl;
109
110     return 0;
111 }
```

Figure 11. Code Snippet of main()

In Figure 11, there is not a lot to explain like how I got the necessary solution since some were just copied from the previous activities and just debugged it to fit the code. A takeaway on this would probably be the new keyword for adding a new object in the heap.

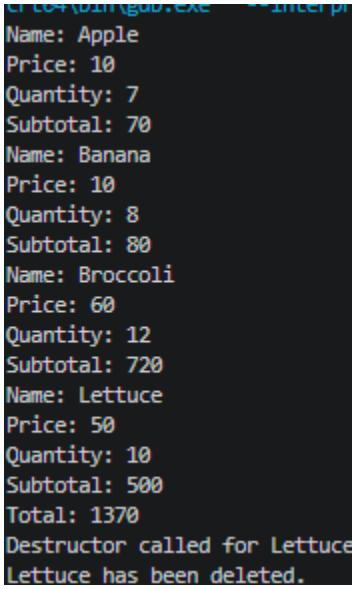


Figure 12. Snippet of Output on Terminal

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Google AI and Grammarly**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Explanation of concepts, better wording on errors, and grammar correction.**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **The AI was used for explaining forgotten and new concepts such as inheritance on C++ and for loops. Just like recommended, I used it to have a better understanding of the errors I'm encountering and pasted the error syntax inside the Google AI. Lastly, Grammarly was used to correct missed commas and some words that should have a capital letter, like Google.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.



Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.